Lesson 9: Organise, test and debug

Pygame CE • S3 onwards • 50-60 min

Learning intention
Make code easier to understand and test by separating repeated tasks.

Success criteria

- I can write and call a function.
- I can test normal and unusual cases.
- I can describe a bug and how I fixed it.

Key ideas

- A **function** packages repeated code under a name.
- Good **test cases** include normal play and edge cases (corners, time at zero, pressing keys together).
- Debugging steps: reproduce, locate, change one thing, retest.

Activities

1. Find repeated or hard-to-read code in your game.
2. Move at least two tasks into functions (for example drawing text or resetting).
3. Work through the testing checklist and record results in a table.
4. Swap with a partner and try to break each other's game.

Code

Example function

```
def draw_text(screen, font, message, position, colour=(255, 255, 255)):
    text = font.render(message, True, colour)
    screen.blit(text, position)

draw_text(screen, font, f"Score: {score}", (10, 10))
draw_text(screen, font, f"Time: {remaining:.0f}", (10, 45))
```

Stretch structure

```
def handle_input(): ...
def update(dt): ...
def draw(): ...

while running:
    dt = clock.tick(60) / 1000
    handle_input()
    update(dt)
    draw()
```

Testing checklist

- Can the player move in every direction?
- Can the player leave the screen?
- Does one collision award exactly one point?
- Does the timer stop scoring?
- Does restart reset every value?

Checkpoint

Learners can describe one bug they found and how they tested the fix.

Support and stretch

- **Support:** Provide a function template to complete.
- **Stretch:** Separate input, update and draw into their own functions.

Reflect

Which test found the most useful problem?

Save your work
Save this version as `lesson9.py` before you start the next lesson.